LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara 21 Dokumentasi endpoint

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

RESTful API merupakan layanan yang memungkinkan aplikasi saling bertukar data melalui protokol HTTP. Dalam pengembangan aplikasi web, API digunakan untuk menjalankan berbagai operasi, seperti mengambil data, menambahkan data, memperbarui data, dan menghapus data.

Selain dapat digunakan oleh aplikasi, endpoint API juga perlu dilengkapi dengan dokumentasi. Dokumentasi endpoint berfungsi sebagai panduan bagi pengembang untuk memahami alamat URL, metode HTTP, parameter, header, format request, serta response yang dihasilkan oleh server. Dengan adanya dokumentasi, proses integrasi antara bagian backend dan frontend dapat dilakukan dengan lebih mudah.

Dokumentasi API yang baik perlu menjelaskan setiap endpoint secara terstruktur, termasuk kebutuhan autentikasi dan kemungkinan kode status HTTP yang dikembalikan. Informasi tersebut membantu pengguna API mengetahui cara mengirimkan permintaan dan memahami hasil yang diberikan oleh server.

Berdasarkan hal tersebut, praktikum Acara 21 membahas pembuatan dokumentasi endpoint pada aplikasi Sistem Informasi Akademik. Dokumentasi dibuat dalam file API_DOCUMENTATION.md dan dilengkapi dengan pemetaan endpoint terhadap Controller dan Method pada aplikasi PHP. Selain itu, Postman digunakan untuk membantu melengkapi dokumentasi API.

1.2 Rumusan Masalah

1. Bagaimana menyusun dokumentasi endpoint pada aplikasi RESTful API?
2. Informasi apa saja yang perlu dicantumkan dalam dokumentasi endpoint?
3. Bagaimana memetakan endpoint dengan Controller dan Method yang digunakan pada aplikasi?
4. Bagaimana memanfaatkan Postman untuk melengkapi dokumentasi API?

1.3 Tujuan Praktikum

1. Memahami fungsi dokumentasi endpoint pada pengembangan RESTful API.

2. Membuat dokumentasi API yang berisi metode HTTP, URL, request, response, dan kode status.
3. Memetakan endpoint ke Controller dan Method pada aplikasi Sistem Informasi Akademik.
4. Memahami penggunaan Postman untuk memberikan deskripsi dan contoh request maupun response API.

BAB 2. LANDASAN TEORI

2.1 RESTful API

RESTful API merupakan antarmuka yang memungkinkan aplikasi saling bertukar data melalui HTTP dengan menggunakan metode seperti GET, POST, PUT, PATCH, dan DELETE. Setiap metode memiliki fungsi yang berbeda, misalnya GET untuk mengambil data dan POST untuk mengirimkan data baru.

RESTful API umumnya menggunakan format JSON untuk mengirimkan dan menerima data sehingga dapat digunakan oleh berbagai aplikasi client.

2.2 Endpoint API

Endpoint merupakan alamat atau URL yang digunakan client untuk mengakses fungsi tertentu pada server. Setiap endpoint memiliki metode HTTP dan tujuan yang berbeda.

Contohnya, GET /mahasiswa digunakan untuk mengambil data mahasiswa, sedangkan POST /mahasiswa digunakan untuk menambahkan data mahasiswa baru.

2.3 Dokumentasi API

Dokumentasi API merupakan penjelasan tertulis mengenai cara menggunakan endpoint yang tersedia pada suatu aplikasi. Dokumentasi membantu pengembang memahami fungsi endpoint tanpa harus membaca seluruh kode program.

Informasi yang umumnya dicantumkan meliputi HTTP Method, URL, header yang diperlukan, parameter, struktur request body, contoh response sukses, contoh response gagal, kode status HTTP, serta keterangan autentikasi.

2.4 HTTP Method dan Status Code

HTTP Method menunjukkan jenis operasi yang dilakukan terhadap suatu resource. Beberapa metode yang umum digunakan adalah GET, POST, PUT, PATCH, dan DELETE.

Sementara itu, HTTP Status Code menunjukkan hasil pemrosesan request oleh server. Beberapa kode status yang umum digunakan antara lain:

Kode	Keterangan
200 OK	Permintaan berhasil diproses.

201 Created	Data berhasil dibuat.
400 Bad Request	Permintaan tidak sesuai.
401 Unauthorized	Autentikasi tidak tersedia atau tidak valid.
404 Not Found	Endpoint atau data tidak ditemukan.
405 Method Not Allowed	Metode HTTP tidak diizinkan.
409 Conflict	Terjadi konflik data.
422 Unprocessable Entity	Data tidak memenuhi validasi.
500 Internal Server Error	Terjadi kesalahan pada server.

2.5 JSON

JavaScript Object Notation (JSON) merupakan format pertukaran data yang umum digunakan pada RESTful API. JSON menyimpan data dalam bentuk pasangan key dan value serta dapat digunakan untuk mengirimkan request body maupun response dari server.

2.6 Postman

Postman merupakan aplikasi yang digunakan untuk mengirim request HTTP dan memeriksa response API. Selain untuk pengujian, Postman menyediakan fitur dokumentasi collection.

Deskripsi pada setiap request dan contoh response dapat disimpan agar pengguna lain memahami cara menggunakan endpoint yang tersedia.

2.7 Arsitektur MVC

Model-View-Controller (MVC) merupakan pola pemisahan bagian aplikasi. Model menangani data dan komunikasi dengan basis data, Controller mengatur proses permintaan, sedangkan Router mengarahkan URL dan metode HTTP ke Controller yang sesuai.

Pada praktikum ini, dokumentasi endpoint mencantumkan hubungan antara URL, Controller, dan Method yang menangani request.

2.8 Postman

Postman merupakan aplikasi untuk mengirim request HTTP dan memeriksa respons API. Postman dapat digunakan untuk menguji proses login, melihat token

yang dihasilkan, menambahkan header Authorization, serta memeriksa status HTTP dan pesan kesalahan.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktikum

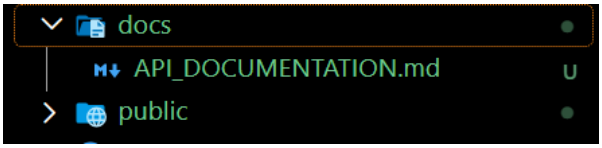
3.1.1 Studi Kasus

Praktikum Acara 21 membahas dokumentasi endpoint API pada Sistem Informasi Akademik. Dokumentasi dibuat untuk menjelaskan fungsi setiap endpoint, Controller yang menangani, serta format request dan response yang digunakan. Proses dokumentasi dilakukan menggunakan file Markdown (API_DOCUMENTATION.md) dan fitur dokumentasi pada Postman.

A. Langkah 1 — Membuat Struktur Folder Dokumentasi

Pada langkah pertama, dibuat folder docs di dalam direktori utama proyek Sistem Informasi Akademik. Folder tersebut digunakan untuk menyimpan file dokumentasi API dengan nama `API_DOCUMENTATION.md`.

Struktur folder yang dibuat adalah sebagai berikut:



B. Langkah 2 — Mendokumentasikan Peta Router

Pada langkah kedua, dibuat tabel pemetaan Router yang berisi alamat URL (resource), Controller yang menangani, serta keterangan fungsi dari setiap endpoint. Peta Router bertujuan untuk memberikan gambaran mengenai hubungan antara URL dengan Controller sebelum menjelaskan setiap endpoint secara terperinci.

```
**Pemetaan resource pada `Router`:**

| Resource (URL) | Controller | Keterangan |
|----------------|------------|------------|
| `/mahasiswa` | `MahasiswaController` | CRUD data mahasiswa (index/store/update/destroy) |
| `/buku` | `BukuController` | CRUD data buku |
| `/login` | `AuthController::login` | Autentikasi pengguna, menghasilkan token |
| `/register` | `AuthController::register` | Pendaftaran pengguna baru |
```

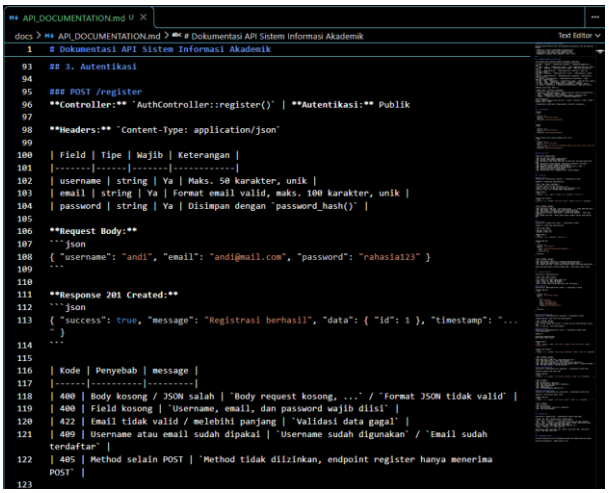
C. Langkah 3 — Menyusun Dokumentasi Endpoint Autentikasi

Pada langkah ketiga, dilakukan dokumentasi endpoint autentikasi yang terdapat pada AuthController, yaitu endpoint Register dan Login. Dokumentasi mencakup HTTP method, alamat endpoint, Controller yang menangani, header, request body, serta response yang dihasilkan.

a) Dokumentasi Endpoint Register

Endpoint Register digunakan untuk melakukan pendaftaran pengguna baru. Endpoint ini ditangani oleh AuthController::register() dengan menggunakan method POST.

- Screenshot file `API_DOCUMENTATION.md`



```
# Dokumentasi API Sistem Informasi Akademik

## 3. Autentikasi

### POST /register
**Controller:** `AuthController::register()` | **Autentikasi:** Publik

**Headers:** `Content-Type: application/json`

| Field | Tipe | Wajib | Keterangan |
|---|---|---|---|
| username | string | Ya | Maks. 50 karakter, unik |
| email | string | Ya | Format email valid, maks. 100 karakter, unik |
| password | string | Ya | Disimpan dengan `password_hash()` |

**Request Body:**
```json
{ "username": "andi", "email": "andi@mail.com", "password": "rahasia123" }
```

Response 201 Created:
```json
{ "success": true, "message": "Registrasi berhasil", "data": { "id": 1 }, "timestamp": "..." }
```

Kode	Penyebab	message
400	Body kosong / JSON salah	`Body request kosong, ...` / `Format JSON tidak valid`
400	Field kosong	`Username, email, dan password wajib diisi`
422	Email tidak valid / melebihi panjang	`Validasi data gagal`
409	Username atau email sudah dipakai	`Username sudah digunakan` / `Email sudah terdaftar`
405	Method selain POST	`Method tidak diizinkan, endpoint register hanya menerima POST`
```

- Kode `AuthController.php`

```php
public function register(): void {
        $input = $this->readJson();

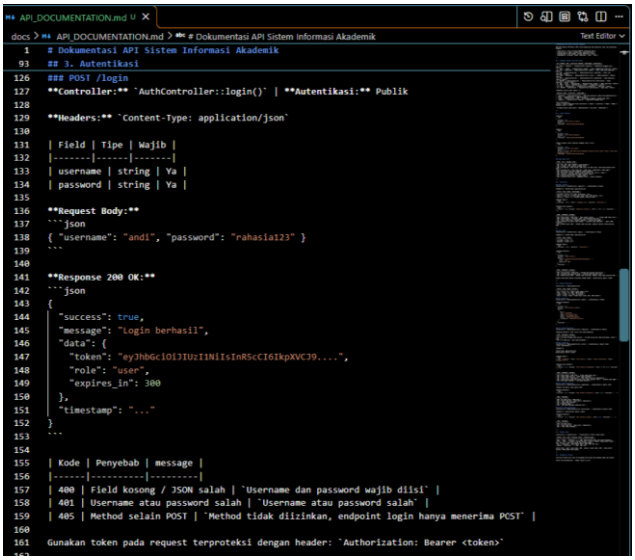
        $data = [
            'username' => $this->str($input, 'username'),
            'email'    => $this->str($input, 'email'),
            // password tidak di-trim: spasi di awal/akhir bisa jadi bagian dari kata sandi
            'password' => isset($input['password']) && is_scalar($input['password']) ? (string) $input['password'] : '',
```

```
        ];
```

b) Dokumentasi Endpoint Login

Endpoint Login digunakan untuk melakukan autentikasi pengguna menggunakan username dan password. Endpoint ini ditangani oleh AuthController::login() dengan menggunakan method POST.

- Screenshot file `API_DOCUMENTATION.md`



- Kode `AuthController.php`

```php
public function login(): void {
        $input = $this->readJson();

        $username = $this->str($input, 'username');
        $password = isset($input['password']) &&
is_scalar($input['password']) ? (string)
$input['password'] : '';

        $missing = [];
        if ($username === '') $missing['username'] =
'Field username wajib diisi';
        if ($password === '') $missing['password'] =
'Field password wajib diisi';
        if ($missing) {
            Response::json(400, false, 'Username dan
password wajib diisi', null, $missing);
```

```
        }

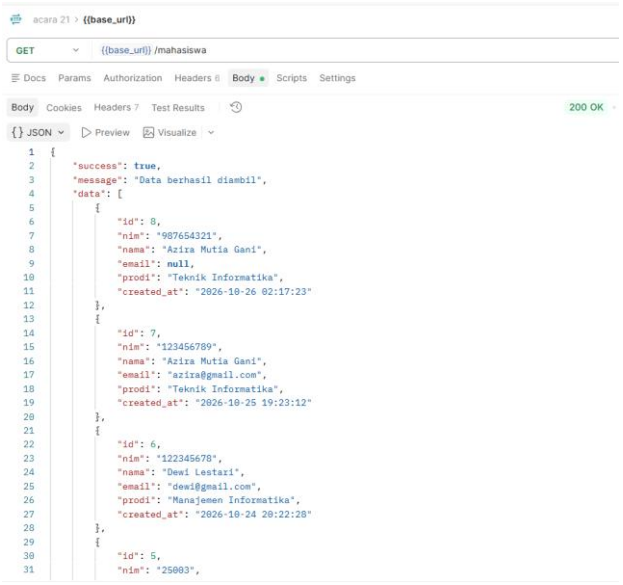
        $user =
$this->model->findByUsername($username);
```

D. Langkah 4 — Menyusun Dokumentasi Endpoint Mahasiswa

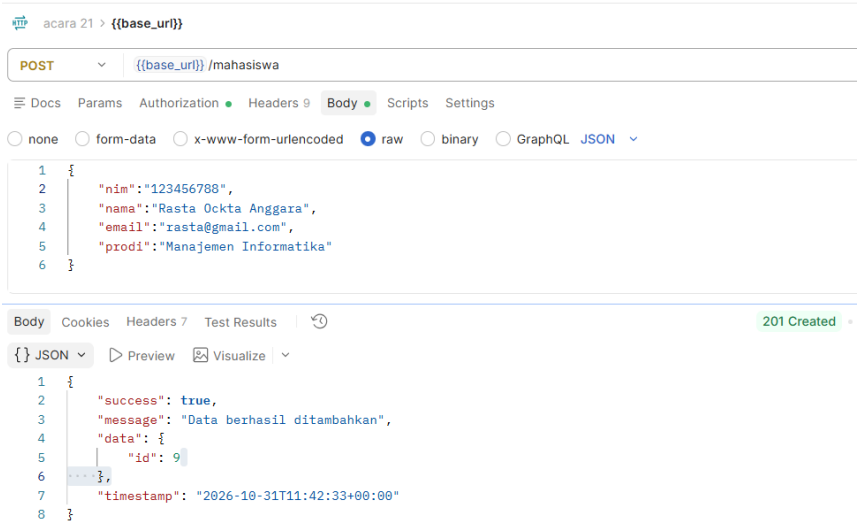
Pada langkah keempat, dilakukan dokumentasi endpoint mahasiswa berdasarkan setiap method yang terdapat pada MahasiswaController. Dokumentasi mencakup operasi pengambilan, penambahan, perubahan, dan penghapusan data mahasiswa.

Endpoint yang didokumentasikan meliputi:

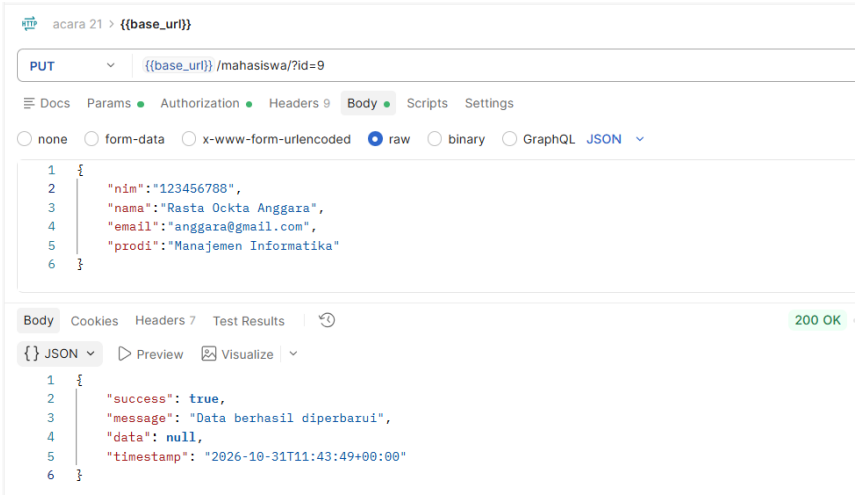
Method	Endpoint	Fungsi
GET	/mahasiswa	Mengambil data mahasiswa



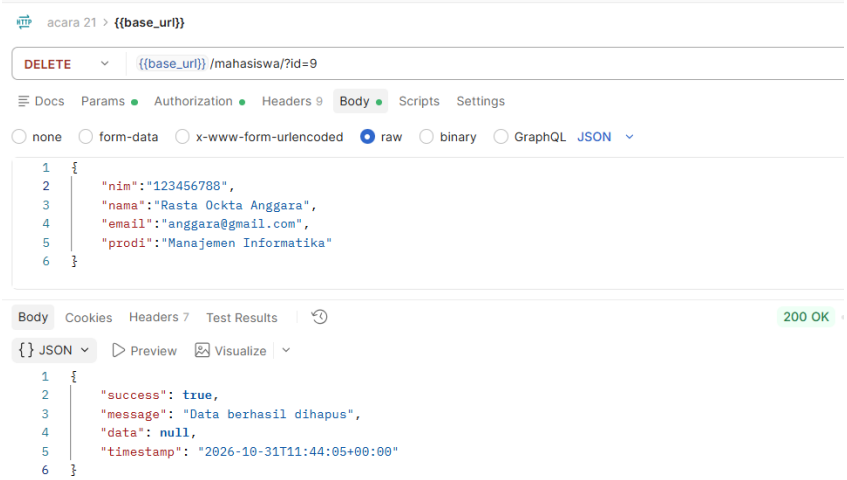
POST	/mahasiswa	Menambahkan data mahasiswa



PUT /mahasiswa Memperbarui data mahasiswa



DELETE /mahasiswa Menghapus data mahasiswa



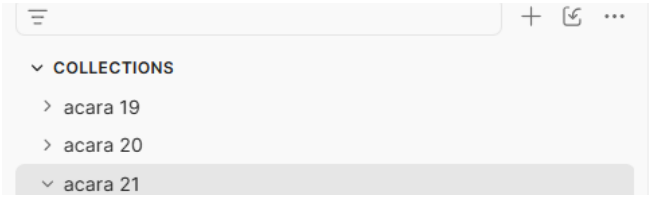
Pada endpoint penambahan, perubahan, dan penghapusan data mahasiswa, diperlukan header Authorization: Bearer <token> sebagai autentikasi sesuai mekanisme Auth::requireAuth().

E. Langkah 5 — Mendokumentasikan melalui Fitur Postman

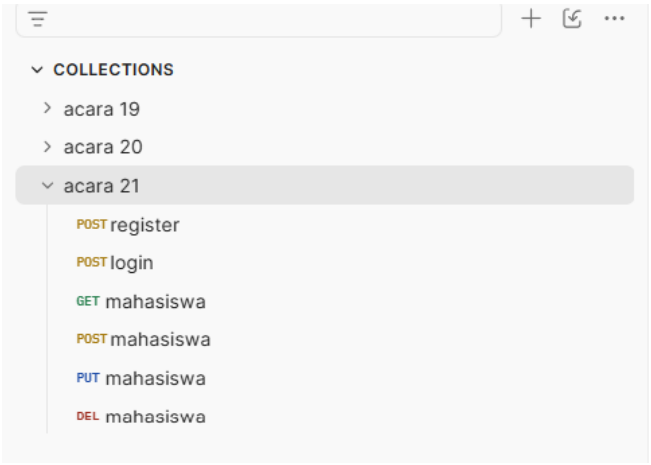
Pada langkah ini dilakukan pendokumentasian seluruh endpoint API menggunakan aplikasi Postman. Dokumentasi bertujuan untuk memberikan informasi mengenai fungsi setiap endpoint, metode HTTP yang digunakan, serta Controller yang menangani setiap request.

a) Membuat Collection Postman

Pertama, dibuat Collection baru dengan nama `acara 21` pada aplikasi Postman. Collection ini digunakan untuk mengelompokkan seluruh endpoint API Si-Akademik agar lebih terstruktur dan mudah dikelola.



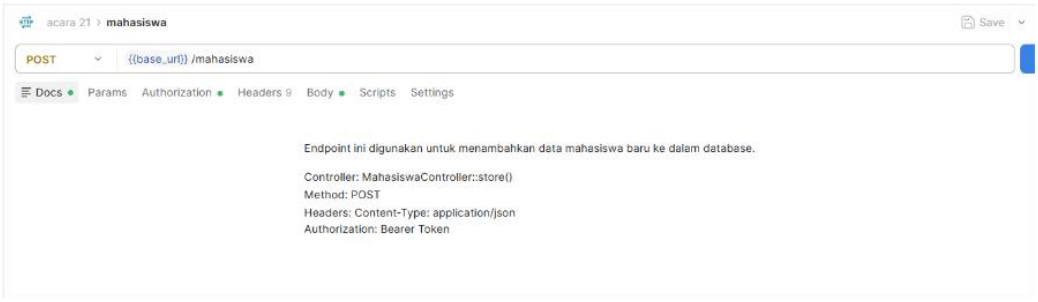
b) Menambahkan Request ke dalam Collection



c) Mengisi Dokumentasi Request

Setiap request dilengkapi dengan Description yang berisi penjelasan fungsi endpoint, nama Controller, method HTTP, serta informasi autentikasi apabila diperlukan. Dengan demikian, setiap

endpoint memiliki keterangan yang memudahkan pengguna memahami cara penggunaannya.

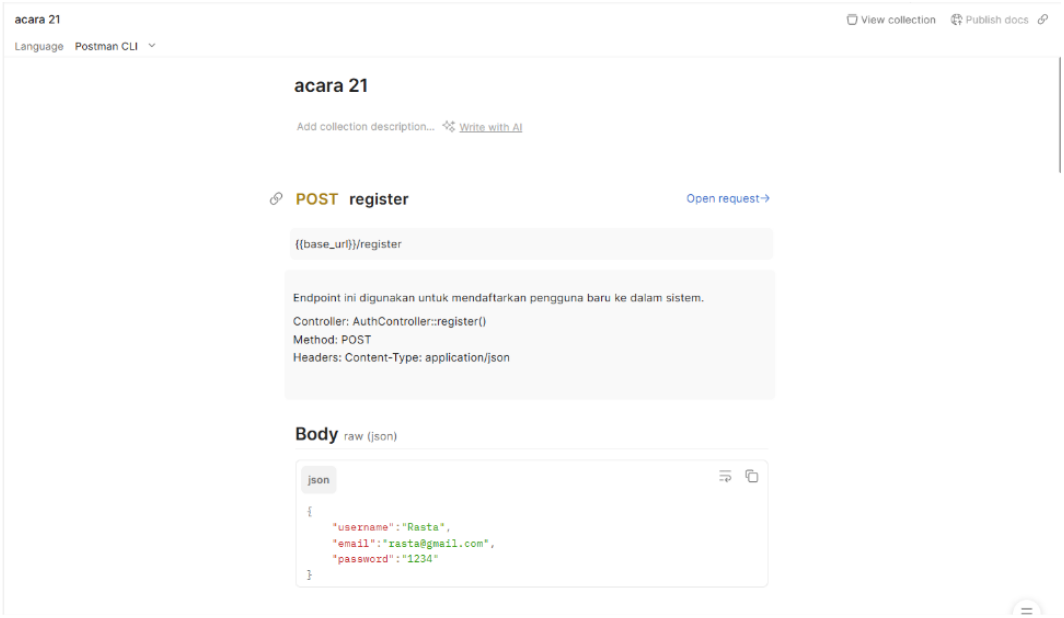


d) Menyimpan Contoh Response

Setelah melakukan pengujian request, response yang diperoleh digunakan sebagai contoh hasil pengujian. Contoh response berhasil maupun gagal dapat disimpan melalui fitur **Save as Example** pada Postman sebagai dokumentasi hasil pengujian API.

e) Menampilkan Dokumentasi Collection

Setelah seluruh request dan Description dilengkapi, dokumentasi Collection ditampilkan melalui menu Overview dengan memilih **View complete documentation**. Halaman tersebut menampilkan dokumentasi endpoint yang telah dibuat dalam satu tampilan sehingga lebih mudah dibaca dan dipahami.



acara 21

View collection Publish docs

Language Postman CLI

POST login

Open request→

{{base_url}}/login

Endpoint ini digunakan untuk melakukan login pengguna ke dalam sistem dan mendapatkan token autentikasi.

Controller: AuthController::login()
Method: POST
Headers: Content-Type: application/json
Response: Token JWT untuk autentikasi.

Body raw (json)

json

```
{
    "username":"Rasta",
    "password":"1234"
}
```

acara 21

View collection Publish docs

Language Postman CLI

GET mahasiswa

Open request→

{{base_url}}/mahasiswa

Endpoint ini digunakan untuk menampilkan seluruh data mahasiswa dari database.

Controller: MahasiswaController::index()
Method: GET
Authorization: Tidak memerlukan Bearer Token (sesuaikan dengan konfigurasi aplikasi).

Authorization Bearer Token

Token	{{json_web_token_0mo5}}

acara 21

View collection Publish docs

Language Postman CLI

POST mahasiswa

Open request→

{{base_url}}/mahasiswa

Endpoint ini digunakan untuk menambahkan data mahasiswa baru ke dalam database.

Controller: MahasiswaController::store()
Method: POST
Headers: Content-Type: application/json
Authorization: Bearer Token

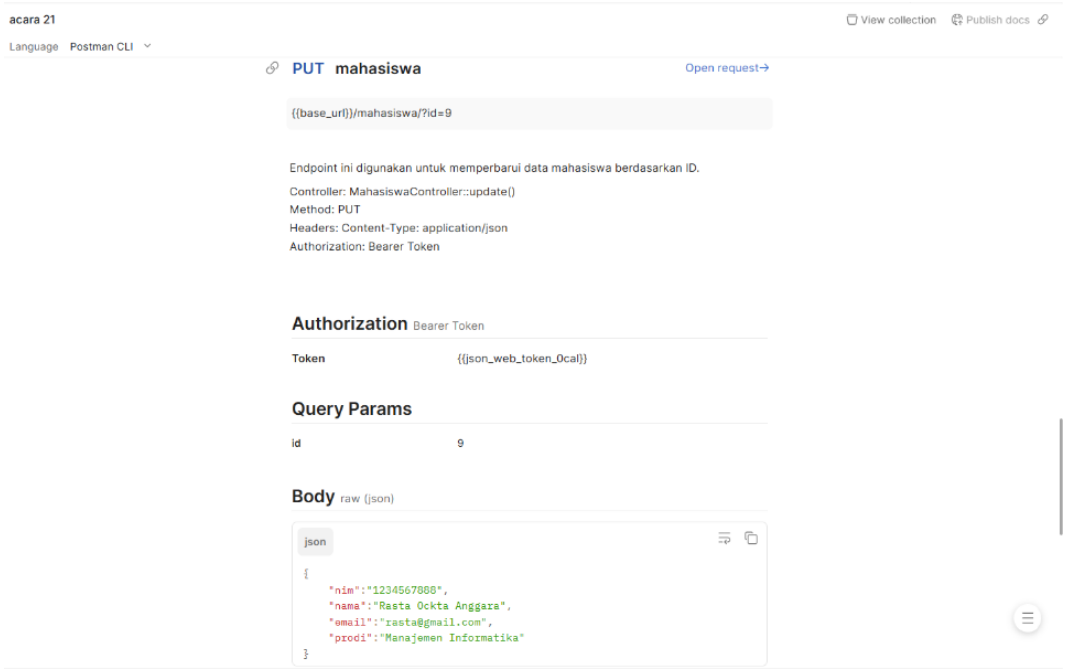
Authorization Bearer Token

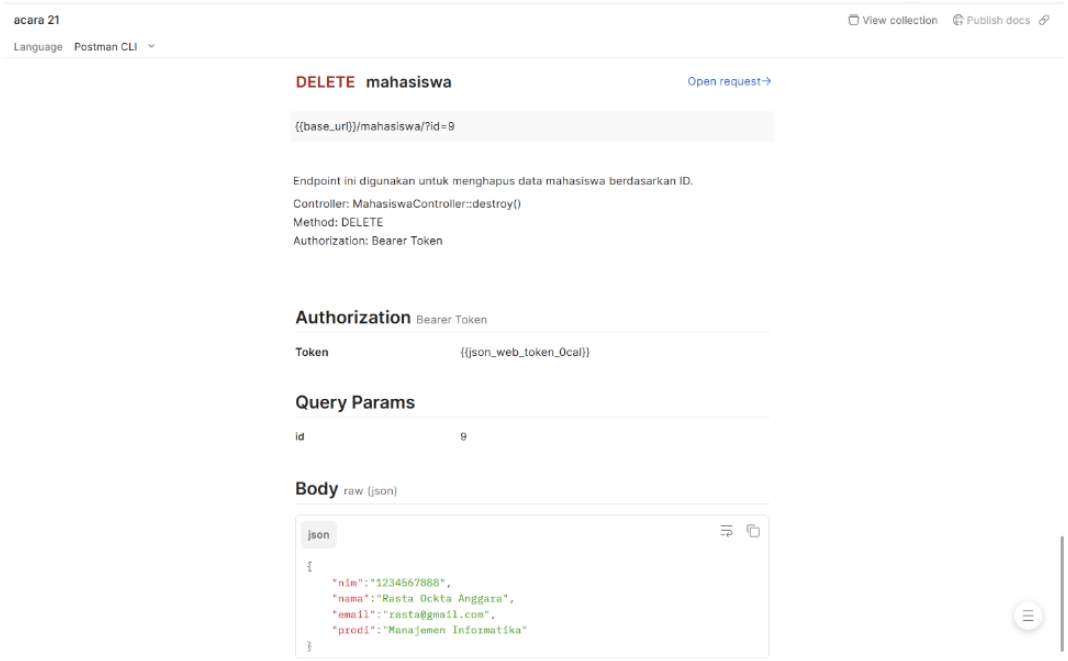
Token	{{json_web_token_0cal}}

Body raw (json)

json

```
{
    "nim":"1234567888",
    "nama":"Rasta Ockta Anggara",
    "email":"rasta@gmail.com",
    "prodi":"Manajemen Informatika"
}
```





3.1.2 Tugas Praktik Mandiri

Pada tugas praktik mandiri dilakukan penyusunan dokumentasi dan penyalinan kode program API pada Sistem Informasi Akademik. Kode program yang digunakan mencakup proses autentikasi dan pengelolaan data mahasiswa. Berikut merupakan beberapa kode program yang digunakan dalam implementasi API.

A. Kode Program Router

```
<?php
```

```php
class Router {

    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',
        'register'  => 'AuthController',
        'login'     => 'AuthController',
    ];

    public function dispatch(): void {
        try {
            $uri = trim(parse_url($_SERVER['REQUEST_URI'],
PHP_URL_PATH), '/');
            $segments = $uri === '' ? [] : explode('/',
$uri);

            // Cari nama resource pada URL; segmen
sesudahnya (jika ada) dianggap ID
            $resource = null;
            $pathId = null;
            foreach ($segments as $i => $segment) {
                if (isset($this->map[$segment])) {
                    $resource = $segment;
                    $pathId = $segments[$i + 1] ?? null;
                    break;
                }
            }

            if ($resource === null) {
                Response::json(404, false, 'Endpoint tidak
ditemukan');
            }

            $id = $pathId ?? ($_GET['id'] ?? null);
            $controllerName = $this->map[$resource];
            $controller = new $controllerName();

            // login bukan resource CRUD: hanya menerima
POST (Acara 20)
            if ($resource === 'login') {
                if ($_SERVER['REQUEST_METHOD'] !== 'POST')
{
                    header('Allow: POST');
                    Response::json(405, false, 'Method
tidak diizinkan, endpoint login hanya menerima POST');
                }
                $controller->login();
```

```
                return;
            }

            // register bukan resource CRUD: hanya menerima
POST
            if ($resource === 'register') {
                if ($_SERVER['REQUEST_METHOD'] !== 'POST')
{
                    header('Allow: POST');
                    Response::json(405, false, 'Method
tidak diizinkan, endpoint register hanya menerima POST');
                }
                $controller->register();
                return;
            }

            switch ($_SERVER['REQUEST_METHOD']) {
                case 'GET':    $controller->index($id);
break;
                case 'POST':   $controller->store(); break;
                case 'PUT':
                case 'PATCH':  $controller->update($id);
break;
                case 'DELETE': $controller->destroy($id);
break;
                default:
                    header('Allow: GET, POST, PUT, PATCH,
DELETE');
                    Response::json(405, false, 'Method
tidak diizinkan');
            }
        } catch (Throwable $e) {
            error_log($e->getMessage());
            Response::json(500, false, 'Terjadi kesalahan
pada server');
        }
    }
}
```

B. Kode Program MahasiswaController

```
<?php

class MahasiswaController extends BaseController
{
    private const NIM_LENGTH = 9;
```

```php
    private Mahasiswa $model;

    public function __construct()
    {
        $this->model = new Mahasiswa();
    }

    // GET /mahasiswa atau GET /mahasiswa?id=1
    public function index($id = null): void
    {
        if ($id !== null && $id !== '') {
            $id = $this->requireId($id);
            $data = $this->model->find($id);

            if (!$data) {
                Response::json(404, false, 'Data tidak
ditemukan');
            }

            Response::json(200, true, 'Data berhasil
diambil', $data);
        }

        Response::json(
            200,
            true,
            'Data berhasil diambil',
            $this->model->getAll()
        );
    }

    // POST /mahasiswa
    public function store(): void
    {
        Auth::requireAuth();

        $data = $this->clean($this->readJson());

        try {
            $id = $this->model->create($data);
        } catch (DomainException $e) {
            Response::json(409, false, $e->getMessage());
        }

        Response::json(
            201,
            true,
```

```
            'Data berhasil ditambahkan',
            ['id' => (int) $id]
        );
    }

    // PUT /mahasiswa?id=1
    public function update($id = null): void
    {
        Auth::requireAuth();

        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            Response::json(404, false, 'Data tidak
ditemukan');
        }

        $data = $this->clean($this->readJson());

        try {
            $this->model->update($id, $data);
        } catch (DomainException $e) {
            Response::json(409, false, $e->getMessage());
        }

        Response::json(200, true, 'Data berhasil
diperbarui');
    }

    // DELETE /mahasiswa?id=1
    public function destroy($id = null): void
    {
        Auth::requireAuth();

        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            Response::json(404, false, 'Data tidak
ditemukan');
        }

        $this->model->delete($id);

        Response::json(200, true, 'Data berhasil dihapus');
    }

    // Membersihkan dan memvalidasi data
```

```
    private function clean(array $input): array
    {
        $data = [
            'nim'   => $this->str($input, 'nim'),
            'nama'  => $this->str($input, 'nama'),
            'email' => $this->str($input, 'email'),
            'prodi' => $this->str($input, 'prodi'),
        ];

        // Tahap 1: field wajib kosong -> HTTP 422
        $missing = [];

        foreach (['nim', 'nama', 'prodi'] as $field) {
            if ($data[$field] === '') {
                $missing[$field] = "Field $field wajib
diisi";
            }
        }

        if (!empty($missing)) {
            Response::json(
                422,
                false,
                'Validasi data gagal',
                null,
                $missing
            );
        }

        // Tahap 2: validasi format data -> HTTP 422
        $errors = [];

        // NIM hanya boleh berisi angka dan harus tepat 9
digit
        if (!preg_match('/^[0-9]{9}$/D', $data['nim'])) {
            $errors['nim'] =
                'NIM hanya boleh mengandung angka dan harus
tepat 9 digit, tidak boleh mengandung huruf';
        }

        // Nama maksimal 100 karakter
        if ($this->len($data['nama']) > 100) {
            $errors['nama'] = 'Nama maksimal 100 karakter';
        }

        // Prodi maksimal 50 karakter
        if ($this->len($data['prodi']) > 50) {
```

```
            $errors['prodi'] = 'Prodi maksimal 50
karakter';
        }

        // Email opsional, tetapi harus valid jika diisi
        if ($data['email'] !== '') {
            if (
                !filter_var($data['email'],
FILTER_VALIDATE_EMAIL) ||
                $this->len($data['email']) > 100
            ) {
                $errors['email'] =
                    'Format email tidak valid atau melebihi
100 karakter';
            }
        } else {
            $data['email'] = null;
        }

        // Kirim seluruh kesalahan validasi
        if (!empty($errors)) {
            Response::json(
                422,
                false,
                'Validasi data gagal',
                null,
                $errors
            );
        }

        return $data;
    }
}
```

C. Kode Program MahasiswaModel

```
<?php
class Mahasiswa {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    public function getAll(): array {
        return $this->db->query('SELECT * FROM mahasiswa
ORDER BY id DESC')->fetchAll();
```

```
    }

    public function find(int $id) {
        $stmt = $this->db->prepare('SELECT * FROM mahasiswa
WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->fetch();
    }

    public function nimExists(string $nim, ?int $exceptId =
null): bool {
        $sql = 'SELECT COUNT(*) FROM mahasiswa WHERE nim
= :nim';
        $params = [':nim' => $nim];
        if ($exceptId !== null) {
            $sql .= ' AND id <> :id';
            $params[':id'] = $exceptId;
        }
        $stmt = $this->db->prepare($sql);
        $stmt->execute($params);
        return (int) $stmt->fetchColumn() > 0;
    }

    public function create(array $data): string {
        if ($this->nimExists($data['nim'])) {
            throw new DomainException('NIM sudah
terdaftar');
        }
        try {
            $stmt = $this->db->prepare(
                'INSERT INTO mahasiswa (nim, nama, prodi,
email) VALUES (:nim, :nama, :prodi, :email)'
            );
            $stmt->execute([
                ':nim'   => $data['nim'],
                ':nama'  => $data['nama'],
                ':prodi' => $data['prodi'],
                ':email' => $data['email'] ?? null,
            ]);
            return $this->db->lastInsertId();
        } catch (PDOException $e) {
            error_log($e->getMessage());
            Response::json(500, false, 'Terjadi kesalahan
pada server');
        }
    }
```

```
    public function update(int $id, array $data): void {
        if ($this->nimExists($data['nim'], $id)) {
            throw new DomainException('NIM sudah digunakan
mahasiswa lain');
        }
        try {
            $stmt = $this->db->prepare(
                'UPDATE mahasiswa SET nim = :nim, nama
= :nama, prodi = :prodi, email = :email WHERE id = :id'
            );
            $stmt->execute([
                ':nim'   => $data['nim'],
                ':nama'  => $data['nama'],
                ':prodi' => $data['prodi'],
                ':email' => $data['email'] ?? null,
                ':id'    => $id,
            ]);
        } catch (PDOException $e) {
            error_log($e->getMessage());
            Response::json(500, false, 'Terjadi kesalahan
pada server');
        }
    }

    public function delete(int $id): void {
        try {
            $stmt = $this->db->prepare('DELETE FROM
mahasiswa WHERE id = :id');
            $stmt->execute([':id' => $id]);
        } catch (PDOException $e) {
            error_log($e->getMessage());
            Response::json(500, false, 'Terjadi kesalahan
pada server');
        }
    }
}
```

D. Kode Program Response

```
<?php
class Response {

    public static function json(int $httpCode, bool
$success, string $message, $data = null, array $errors =
[]): void {
        http_response_code($httpCode);
        header('Content-Type: application/json;
charset=utf-8');
```

```
        $body = [
            'success' => $success,
            'message' => $message,
            'data'    => $data,
        ];

        // Field errors hanya muncul jika ada kesalahan
validasi
        if (!empty($errors)) {
            $body['errors'] = $errors;
        }

        $body['timestamp'] = date('c');

        $flags = JSON_UNESCAPED_SLASHES |
JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE;

        // Tugas Mandiri 2: PRETTY_PRINT hanya pada mode
pengembangan
        if (self::isDevelopment()) {
            $flags |= JSON_PRETTY_PRINT;
        }

        echo json_encode($body, $flags);
        exit;
    }

    private static function isDevelopment(): bool {
        return defined('APP_ENV') && APP_ENV ===
'development';
    }
}
```

E. . Dokumentasi API

```
    # Dokumentasi API Sistem Informasi Akademik

REST API berbasis PHP Native (MVC) untuk pengelolaan data
mahasiswa, buku, dan autentikasi pengguna.

- **Base URL:** `http://localhost/BKPM/si-
akademik21/public`
- **Format data:** JSON (`Content-Type: application/json`)
- **Autentikasi:** JWT (HS256), berlaku 300 detik (5
menit)
- **Database:** `workshop` (tabel `mahasiswa`, `buku`,
`users`)
```

```
---

## 1. Ringkasan Endpoint dan Peta Router

| No | Method | URL | Controller::Method | Keterangan | Autentikasi |
|----|--------|-----|--------------------|------------|-------------|
| 1 | POST | `/register` | `AuthController::register()` | Pendaftaran pengguna baru | Publik |
| 2 | POST | `/login` | `AuthController::login()` | Login, menghasilkan token JWT | Publik |
| 3 | GET | `/mahasiswa` | `MahasiswaController::index()` | Ambil semua data mahasiswa | Publik |
| 4 | GET | `/mahasiswa?id={id}` | `MahasiswaController::index($id)` | Ambil satu mahasiswa | Publik |
| 5 | POST | `/mahasiswa` | `MahasiswaController::store()` | Tambah mahasiswa | **Bearer Token** |
| 6 | PUT | `/mahasiswa?id={id}` | `MahasiswaController::update($id)` | Ubah mahasiswa | **Bearer Token** |
| 7 | DELETE | `/mahasiswa?id={id}` | `MahasiswaController::destroy($id)` | Hapus mahasiswa | **Bearer Token** |
| 8 | GET | `/buku`, `/buku?id={id}` | `BukuController::index()` | Ambil data buku | Publik |
| 9 | POST | `/buku` | `BukuController::store()` | Tambah buku | Publik |
| 10 | PUT | `/buku?id={id}` | `BukuController::update($id)` | Ubah buku | Publik |
| 11 | DELETE | `/buku?id={id}` | `BukuController::destroy($id)` | Hapus buku | Publik |

**Pemetaan resource pada `Router`:**

| Resource (URL) | Controller | Keterangan |
|----------------|------------|------------|
| `/mahasiswa` | `MahasiswaController` | CRUD data mahasiswa (index/store/update/destroy) |
| `/buku` | `BukuController` | CRUD data buku |
| `/login` | `AuthController::login` | Autentikasi pengguna, menghasilkan token |
| `/register` | `AuthController::register` | Pendaftaran pengguna baru |
```

Alur permintaan:
`Client -> public/index.php (Front Controller) -> Router -> Controller -> Model -> MySQL -> Response::json() -> JSON`

> ID dapat dikirim lewat query (`/mahasiswa?id=1`) atau path (`/mahasiswa/1`).

---

2. Format Response

Sukses
```json
{
  "success": true,
  "message": "Data berhasil diambil",
  "data": {},
  "timestamp": "2026-10-30T10:00:00+00:00"
}
```

Gagal
```json
{
  "success": false,
  "message": "Data tidak ditemukan",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+00:00"
}
```

Gagal validasi (422) memiliki tambahan field `errors`:
```json
{
  "success": false,
  "message": "Validasi data gagal",
  "data": null,
  "errors": { "nim": "NIM hanya boleh mengandung angka dan harus tepat 9 digit, tidak boleh mengandung huruf" },
  "timestamp": "2026-10-30T10:00:00+00:00"
}
```

Kode Status HTTP

| Kode | Arti | Dipakai untuk |

```
|------|------|---------------|
| 200 | OK | GET / PUT / DELETE / login berhasil |
| 201 | Created | POST berhasil membuat data |
| 400 | Bad Request | Body kosong, JSON rusak, ID tidak
valid, field wajib kosong (auth, buku) |
| 401 | Unauthorized | Token tidak ada / tidak valid /
kedaluwarsa, login gagal |
| 404 | Not Found | Data atau endpoint tidak ditemukan |
| 405 | Method Not Allowed | Method tidak diizinkan (mis.
GET ke `/login`) |
| 409 | Conflict | Data duplikat (NIM, username, email) |
| 422 | Unprocessable Entity | Validasi gagal |
| 500 | Internal Server Error | Kegagalan server / koneksi
database |

---

## 3. Autentikasi

### POST /register
**Controller:** `AuthController::register()` |
**Autentikasi:** Publik

**Headers:** `Content-Type: application/json`

| Field | Tipe | Wajib | Keterangan |
|-------|------|-------|------------|
| username | string | Ya | Maks. 50 karakter, unik |
| email | string | Ya | Format email valid, maks. 100
karakter, unik |
| password | string | Ya | Disimpan dengan
`password_hash()` |

**Request Body:**
```json
{ "username": "andi", "email": "andi@mail.com",
"password": "rahasia123" }
```

Response 201 Created:
```json
{ "success": true, "message": "Registrasi berhasil",
"data": { "id": 1 }, "timestamp": "..." }
```

Kode	Penyebab	message
```

````
| 400 | Body kosong / JSON salah | `Body request kosong, ...` / `Format JSON tidak valid` |
| 400 | Field kosong | `Username, email, dan password wajib diisi` |
| 422 | Email tidak valid / melebihi panjang | `Validasi data gagal` |
| 409 | Username atau email sudah dipakai | `Username sudah digunakan` / `Email sudah terdaftar` |
| 405 | Method selain POST | `Method tidak diizinkan, endpoint register hanya menerima POST` |

---

### POST /login
**Controller:** `AuthController::login()` |
**Autentikasi:** Publik

**Headers:** `Content-Type: application/json`

| Field | Tipe | Wajib |
|-------|------|-------|
| username | string | Ya |
| password | string | Ya |

**Request Body:**
```json
{ "username": "andi", "password": "rahasia123" }
```

**Response 200 OK:**
```json
{
  "success": true,
  "message": "Login berhasil",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....",
    "role": "user",
    "expires_in": 300
  },
  "timestamp": "..."
}
```

| Kode | Penyebab | message |
|------|----------|---------|
| 400 | Field kosong / JSON salah | `Username dan password wajib diisi` |
````

```
| 401 | Username atau password salah | `Username atau password salah` |
| 405 | Method selain POST | `Method tidak diizinkan, endpoint login hanya menerima POST` |

Gunakan token pada request terproteksi dengan header: `Authorization: Bearer <token>`

---

## 4. Endpoint Mahasiswa

**Controller:** `MahasiswaController`

| Field | Tipe | Wajib | Aturan |
|---|---|---|---|
| nim | string | Ya | Tepat 9 digit angka, unik |
| nama | string | Ya | Maks. 100 karakter |
| prodi | string | Ya | Maks. 50 karakter |
| email | string | Tidak | Format email valid, maks. 100 karakter |

### GET /mahasiswa
**Controller:** `MahasiswaController::index()` |
**Autentikasi:** Publik

**Response 200 OK:**
```json
{
  "success": true,
  "message": "Data berhasil diambil",
  "data": [
    {
      "id": 7,
      "nim": "123456789",
      "nama": "Azira Mutia Gani",
      "email": "azira@gmail.com",
      "prodi": "Teknik Informatika",
      "created_at": "2026-10-25 12:23:12"
    }
  ],
  "timestamp": "..."
}
```

GET /mahasiswa?id={id}
```

````
**Controller:** `MahasiswaController::index($id)` |
**Autentikasi:** Publik

**Response 200 OK:** `data` berisi satu objek mahasiswa.

| Kode | Penyebab | message |
|------|----------|---------|
| 400 | ID bukan bilangan bulat positif | `ID tidak valid
atau tidak disertakan, contoh: ?id=1` |
| 404 | ID tidak ada | `Data tidak ditemukan` |

### POST /mahasiswa
**Controller:** `MahasiswaController::store()` |
**Autentikasi:** Bearer Token (`Auth::requireAuth()`)

**Headers:**
```
Content-Type: application/json
Authorization: Bearer <token>
```

**Request Body:**
```json
{ "nim": "220100123", "nama": "Andi Saputra", "prodi":
"Teknik Informatika", "email": "andi@mail.com" }
```

**Response 201 Created:**
```json
{ "success": true, "message": "Data berhasil ditambahkan",
"data": { "id": 8 }, "timestamp": "..." }
```

| Kode | Penyebab | message |
|------|----------|---------|
| 400 | Body kosong / JSON salah | `Format JSON tidak
valid` |
| 401 | Token tidak dikirim | `Token tidak ditemukan` |
| 401 | Token salah / kedaluwarsa | `Token tidak valid
atau kedaluwarsa` |
| 422 | Field wajib kosong atau format salah (NIM,
panjang, email) | `Validasi data gagal` |
| 409 | NIM sudah terdaftar | `NIM sudah terdaftar` |

### PUT /mahasiswa?id={id}
**Controller:** `MahasiswaController::update($id)` |
**Autentikasi:** Bearer Token
````

Headers dan Body: sama seperti POST.

Response 200 OK:
```json
{ "success": true, "message": "Data berhasil diperbarui",
"data": null, "timestamp": "..." }
```

Kode	Penyebab
400	ID tidak valid / JSON salah
401	Token tidak ada / tidak valid / kedaluwarsa
404	`Data tidak ditemukan`
422	Validasi gagal
409	`NIM sudah digunakan mahasiswa lain`

DELETE /mahasiswa?id={id}
Controller: `MahasiswaController::destroy($id)` |
Autentikasi: Bearer Token

Headers: `Authorization: Bearer <token>`

Response 200 OK:
```json
{ "success": true, "message": "Data berhasil dihapus",
"data": null, "timestamp": "..." }
```

Kode	Penyebab
400	ID tidak valid
401	Token tidak ada / tidak valid / kedaluwarsa
404	`Data tidak ditemukan`

---

5. Endpoint Buku

Controller: `BukuController` | **Autentikasi:** Publik (tanpa token)

Method	URL	Body	Response sukses	Response gagal
GET	`/buku`, `/buku?id=1`	-	200	400 ID tidak valid, 404 tidak ditemukan

```
| POST | `/buku` | `{"judul":"Pemrograman
Web","penulis":"Ahmad","tahun_terbit":2024}` | 201,
`data.id` | 400 field kosong, 422 validasi gagal |
| PUT | `/buku?id=1` | sama seperti POST | 200 | 400, 404,
422 |
| DELETE | `/buku?id=1` | - | 200 | 400, 404 |

Aturan field: `judul` wajib (maks. 150), `penulis` wajib
(maks. 100), `tahun_terbit` opsional (1000 sampai tahun
depan).

---

## 6. Dokumentasi Postman

Collection Postman dari Acara 21 dilengkapi Description
dan Examples pada tiap request.

Tautan View Documentation: `<tempel tautan di sini>`
```

BAB 4. PEMBAHASAN

4.1 Pembahasan

Pada kegiatan praktik Acara 21 dilakukan penyusunan dokumentasi endpoint REST API pada Sistem Informasi Akademik menggunakan PHP Native dengan konsep Model-View-Controller (MVC). Dokumentasi disusun dalam file `API_DOCUMENTATION.md` yang berisi informasi mengenai Base URL, format data JSON, autentikasi JWT, daftar endpoint, serta format request dan response.

Proses pengelolaan request API dilakukan melalui Router. Router berfungsi mengarahkan setiap permintaan berdasarkan URL dan metode HTTP menuju Controller yang sesuai. Pada sistem ini, `MahasiswaController` digunakan untuk mengelola data mahasiswa, sedangkan `AuthController` digunakan untuk menangani proses registrasi dan login pengguna.

Endpoint mahasiswa menggunakan beberapa metode HTTP, yaitu GET untuk mengambil data mahasiswa, POST untuk menambahkan data, PUT untuk memperbarui data, dan DELETE untuk menghapus data. Setiap metode memiliki fungsi yang berbeda sesuai dengan kebutuhan pengelolaan data.

Dalam penerapannya, API menggunakan format JSON sebagai media pertukaran data antara client dan server. Autentikasi menggunakan JSON Web Token (JWT) dengan algoritma HS256 dan masa berlaku selama 300 detik atau 5 menit. Token diperlukan ketika mengakses endpoint POST, PUT, dan DELETE mahasiswa, sedangkan endpoint GET, register, dan login dapat diakses secara publik.

Selanjutnya, pengujian endpoint dilakukan menggunakan Postman. Setiap request dikonfigurasi berdasarkan metode HTTP, URL, serta autentikasi yang diperlukan. Melalui pengujian tersebut, response API dapat diamati untuk mengetahui hasil permintaan, baik berupa data yang berhasil diproses maupun pesan kesalahan.

Penyusunan dokumentasi API dan pengujian menggunakan Postman membantu memberikan gambaran mengenai alur komunikasi antara client dan server. Dokumentasi yang terstruktur juga memudahkan pengguna dalam memahami fungsi setiap endpoint serta cara menggunakannya.

BAB 5. PENUTUP

5.1 Kesimpulan

Berdasarkan kegiatan praktik Acara 21, dapat disimpulkan bahwa dokumentasi endpoint API berperan penting dalam menjelaskan penggunaan dan fungsi setiap endpoint pada Sistem Informasi Akademik. Melalui penyusunan file `API_DOCUMENTATION.md`, informasi mengenai metode HTTP, URL, Controller, autentikasi, serta format request dan response dapat disusun secara terstruktur.

Selain itu, penggunaan Router membantu mengarahkan request menuju Controller yang sesuai, sedangkan Postman digunakan untuk melakukan pengujian endpoint API. Kegiatan ini memberikan pemahaman mengenai proses komunikasi antara client dan server serta penerapan dokumentasi REST API menggunakan PHP Native.

5.2 Saran

Dokumentasi API sebaiknya disusun secara lengkap, jelas, dan mudah dipahami agar dapat digunakan sebagai panduan oleh pengembang maupun pengguna. Selain itu, setiap perubahan pada endpoint perlu diikuti dengan pembaruan dokumentasi. Pengujian menggunakan Postman juga perlu dilakukan secara berkala untuk memastikan kesesuaian antara dokumentasi dan implementasi API.